IT2080 IT Project

Year 2, Semester 2, 2025

Project Progress Report

Mobile, Electronics and Service

Group Details

Campus: Malabe Campus

ITP Group Number: ITP25_WE_B1_215

IT Number	Student Name	E mail	Contact Number
IT23571884	Meshech Johnathan P R	It23571884@my.aliit.lk	071 6610 474
IT23313248	K. Mathuran	It23313248@my.sliit.lk	076 0837 551
IT23542242	Thushanya J	It23542242@my.sliit.lk	0752414453
IT23540194	A.A.R.A.M Shakna	IT23540194@my.sliit.lk	0777076743
IT23614758	Thayaparan Danuskanth	IT23614758@my.sliit.lk	0752695486

Table of Contents

1. Progress 5
2. ER Diagram 11
3. Relational Model 12
4. Test Case Design 13
4.1. Order management 13
4.1.1. Search for Electronics Devices 13
4.1.2. Add Device to Cart 13
4.1.3. View Cart and Update Quantity 13
4.1.4. Remove Device from Cart 13
4.1.5. Make a Payment 14
4.1.6. View Order History 14
4.1.7. Track Order Status 14
4.2. Admin Dashboard 15
4.2.1. Check User Details 15
4.2.2. Update User Details 15
4.2.3. Delete User 15
4.2.4. Add Product 15
4.2.5. Check Product List 16
4.2.6. Update Product Details 16
4.2.7. Delete Product 16
4.2.8. Approve/Decline Warranty Claims 16
4.2.9. Approve/Decline Repair Requests 17
4.2.10. Update Order Status 17
4.2.11. View Order List 17

4.3. Repair and solution management 18
4.3.1. Submit Repair Request 18
4.3.2. View own repair requests 18
4.3.3. Update Repair Description or Images 18
4.3.4. Search for repair requests 19
4.3.5. Filter & Sort Repair Requests 19
4.3.6. View Repair Request Details 19
4.3.7. Update Repair Status (Admin Only) 19
4.3.8. Admin filter by status 20
4.3.9. Invalid Repair Request Submission 20
4.3.10. Unauthorized Update Attempt 20
4.4. Warranty Management 21
4.4.1. Submit Warranty Claim 21
4.4.2. View Warranty Claim Status 21
4.4.3. Update or Cancel Warranty Claim 21
4.4.4. View Past Warranty Claims 22
4.4.5. Submit Claim for Unregistered Product 22
4.4.6. Unauthorized Claim Edit Attempt 22
4.4.7. Upload Invalid Proof of Purchase 23
5. High Level system Diagram 26
6. Innovation Parts of the Project 27
6.1. Stock update in real time 27
6.2. Special feature 27
6.2.1. Chat box 27
6.2.2. Google Map Integration 27

6.2.3. OTP Verification 27
6.2.4. JWT (JSON Web Token) Authentication 28
6.2.5. Role-Based Access Control 28
6.2.6. Encrypted Passwords 28
6.2.7. QR Code Integration 28
6.2.8. JSON Web Token (Advanced Session Security) 28
7. Commercialization 29
7.1. Market Overview 29
7.2. Value Proposition 29
7.3. Revenue Model 30
7.4. Product Growth 30
7.5. Implementation Plan 31
7.6. Marketing Plan 31
7.6.1. Regulation and Compliance 32
7.7. Risk Management 32
7.8. Financials 32
7.9. Partnerships 33
7.10. Exit Plan 33
8. Individual Contribution 33
8.1. Order Management (IT23571884) 33
8.1.2. Flow Chart 35
8.1.3. Pseudocode 36
8.2. Admin Dashboard Management 39
8.2.2. Flow Chart 40
8.2.1. Pseudocode 41

8.3. Repair and solution Management (IT23542242) 44
8.3.1. Features 44
8.3.2. Flow Chart 45
8.3.3. Pseudocode 46
8.4. Warranty Management (IT23540194) 49
Warranty Claims 49
Claim History 49
8.4.1. Features 49
8.4.2. Flowchart 50
8.4.3. Pseudocode 51
8.5 User Management (IT23614758) 55
8.5.1 Features 55
8.4.4. Flowchart 56
8.5.3 Pseudocode 57

1. Progress

Function	Completed User stories	Incomplete User Stories
Order management (IT23571884)	• As a customer, I need to search for electronic devices on the website, so that I can easily find the product I want to buy. • As a customer, I should be able to add selected products to my cart, so that I	• Pending customer confirmation for the payment process • Pending Admin update for the order delivery status

	can purchase them later in a single checkout. • As a customer, I should be able to view and update my cart, so that I can change quantities or remove items before payment. • As a customer, I need to proceed with secure payment, so that I can confirm my purchase successfully. • As a customer, I should be able to view my order history, so that I can track past purchases and manage warranties. • As a customer, I should be able to track the status of my order (Processing, Shipped, Delivered), so that I know when to expect my product.	
Admin Dashboard (IT23313248)	• As an admin, I need to view the list of all users, so that I can check their details and monitor system access. • As an admin, I should be able to update user details, so that I can keep their	• Pending admin approval for customer warranty claim verification • Pending admin review for product stock update request

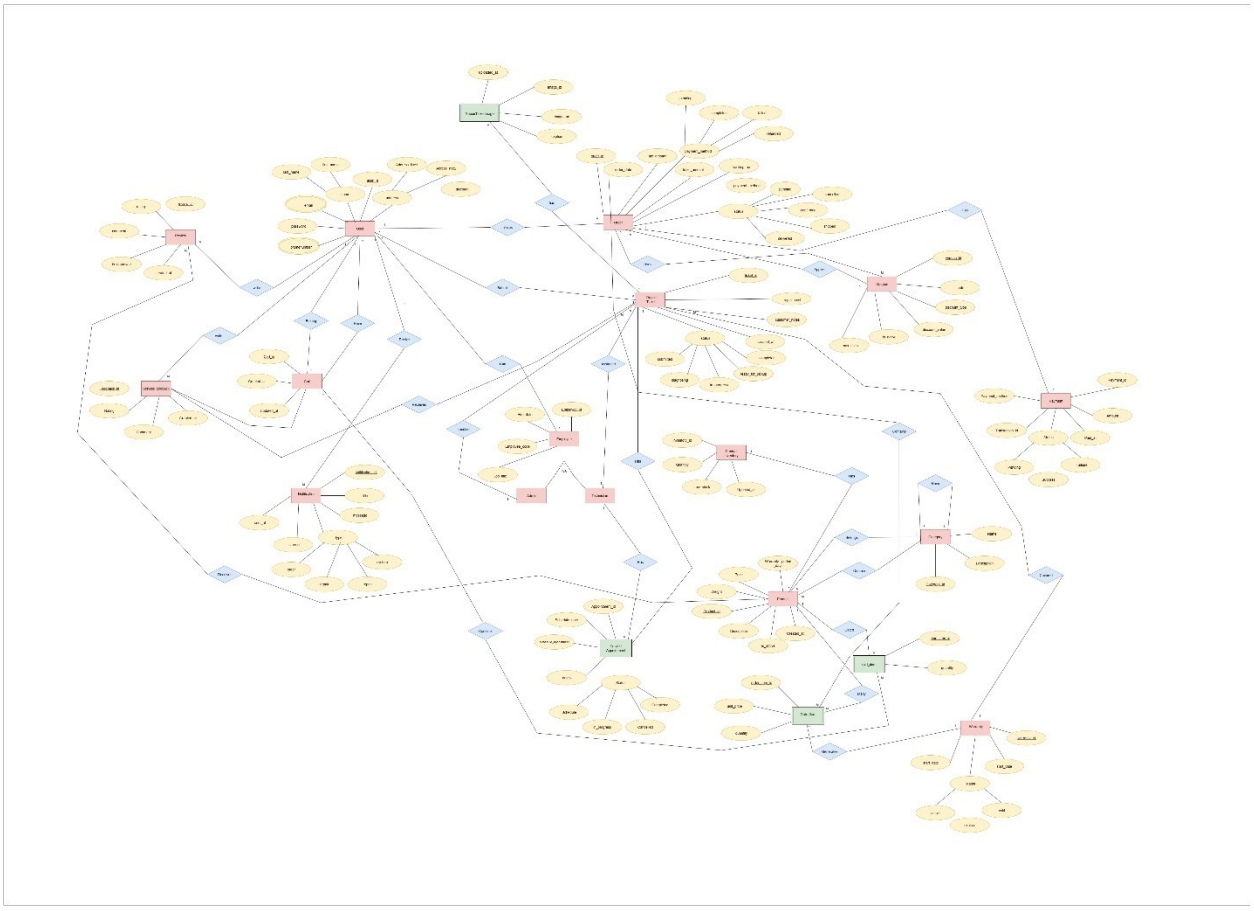
	information accurate and up to date. • As an admin, I should be able to delete user accounts, so that I can remove inactive or unauthorized users. • As an admin, I need to add new products to the system, so that customers can see and purchase them on the website. • As an admin, I should be able to update product details, so that the catalog always reflects current prices, stock, and specifications. • As an admin, I should be able to delete products, so that outdated or unavailable items are removed from the catalog. • As an admin, I need to approve or decline warranty claims, so that customers get quick responses for their service requests. • As an admin, I need to approve or decline repair requests, so that I can	

	control which service requests are valid and actionable. • As an admin, I should be able to update the order status, so that customers can track their delivery progress in real time. • As an admin, I need to view the full order list, so that I can monitor customer purchases and manage order fulfillment.	
Warranty management (IT23540194)	• As a customer, I should be able to submit a warranty claim request, so that I can request repairs or replacements under warranty. • As a customer, I need to upload proof of purchase (such as an invoice or receipt), so that my warranty claim can be validated. • As a customer, I should be able to assess the status of my warranty claims (Pending, Approved, Declined, Completed), so	• Pending customer confirmation for warranty claim submission. • Pending admin approval for warranty claim verification. • Pending system notification to update customers about claim status

	that I know the progress of my request. As a customer, I should receive notifications (email/SMS) about my warranty claim updates, so that I stay informed about approval or rejection.	
Repair and solution management (IT23542242)	• As a user, I want to securely log in and access protected routes so that my data is safe. • As an admin, I want elevated privileges to manage repair statuses and oversee all requests. • As a user, I want to create, view, update, and delete my repair requests. • As a user, I want clear error messages and validation to guide my input. • As an admin, I want to sort repair requests by status, customer name, product name, and date range so that I can quickly analyze and prioritize tasks. • As a user, I want to view a list of all my submitted repair requests so that I can	• As a user, I want a frontend interface to interact with repair requests. • As a user, I want to upload images of damaged and repaired products through the frontend so that I can visually document the repair process. • As an admin, I want to export filtered repair request data as a PDF so that I can generate reports and share summaries.

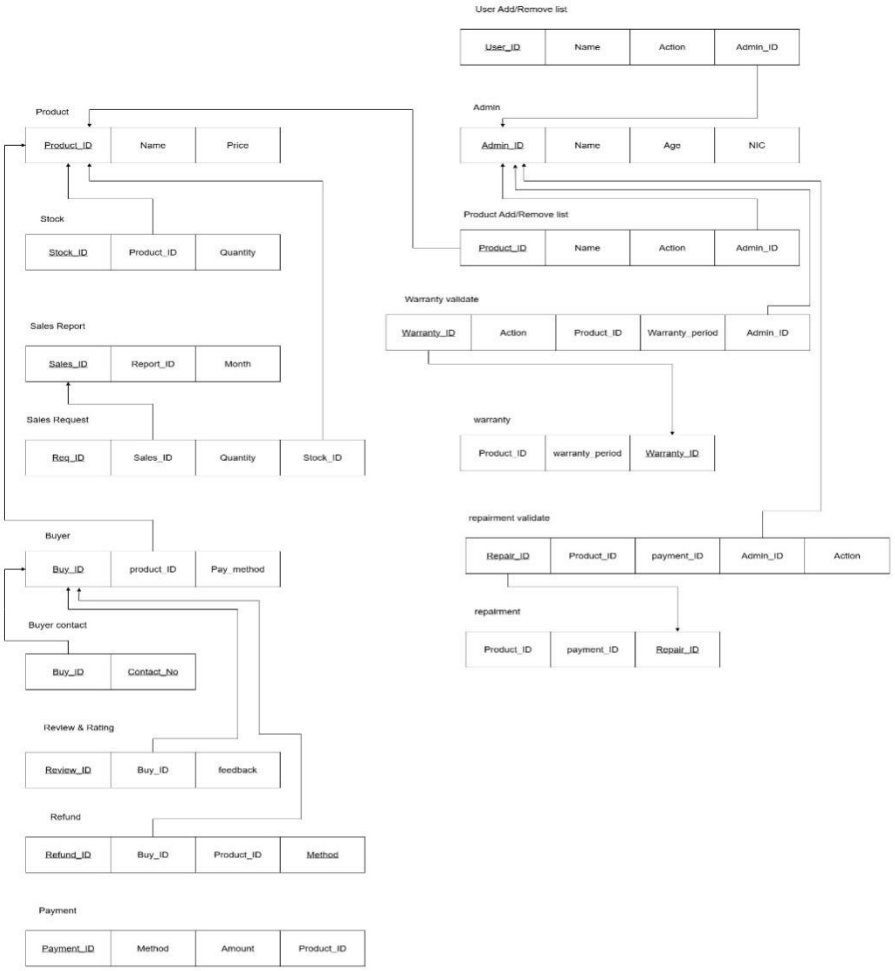
	track their progress and history.	
User management (IT23614758)	• As a user, I want to sign up, log in, and reset my password so that I can securely access my account • As a user, I want the AI to provide financial insights and detect risks so that I can make better business decisions. • As a user, I want an AI chatbot to answer my financial queries so that I can get instant support.	• Users can view, register, and manage multiple mobile/electronic devices linked to their account.

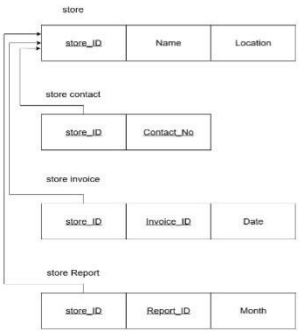
2. ER Diagram



Link – ER-Diagram

3. Relational Model





Link – Relational Model

4. Test Case Design

4.1. Order management

4.1.1. Search for Electronics Devices

- **Scenario** – A user searches for electronic devices in the system.
- **Test Cases**
  - **Step 1** – The user logs into the system and navigates to the search bar.
  - **Test Case 1** – Enter the product/device name (e.g., "iPhone 14") and click the search button.
  - **Result** – The system should display a list of matching products with details like name, price, and availability.

4.1.2. Add Device to Cart

- **Scenario** – A user adds a selected device to their cart.
- **Test Cases**
  - **Step 1** – After searching, the user selects a device from the product list.
  - **Test Case 1** – Click the "Add to Cart" button for the selected device.
  - **Result** – The system should display a confirmation message and update the cart with the selected product.

4.1.3. View Cart and Update Quantity

- **Scenario** – A user views their cart and updates product quantity.
- **Test Cases**
  - **Step 1** – The user navigates to the cart section.
  - **Test Case 1** – Increase/decrease the product quantity and click update.
  - **Result** – The cart should refresh and display the updated quantity with recalculated total price.

4.1.4. Remove Device from Cart

- **Scenario** – A user removes a product from their cart.
- **Test Cases**
  - **Step 1** – The user navigates to the cart section.

- **Test Case 1** – Click the "Remove" or "Delete" button beside a product.
- **Result** – The system should remove the item and update the cart total accordingly.

4.1.5. Make a Payment

- **Scenario** – A user proceeds to checkout and completes payment.
- **Test Cases**
  - **Step 1** – The user clicks "Proceed to Checkout" from the cart.
  - **Test Case 1** – Enter shipping address, select payment method (card, UPI, etc.), and click pay.
  - **Result** – The system should process the payment, generate an order confirmation, and provide an order ID.

4.1.6. View Order History

- **Scenario** – A user views their past orders.
- **Test Cases**
  - **Step 1** – The user navigates to the "My Orders" section.
  - **Test Case 1** – Click on the order history tab.
  - **Result** – The system should display all previous orders with details (order ID, product name, status, and payment info).

4.1.7. Track Order Status

- **Scenario** – A user checks the delivery status of a placed order.
- **Test Cases**
  - **Step 1** – The user navigates to the "My Orders" section and selects an order.
  - **Test Case 1** – Click on the order tracking button.
  - **Result** – The system should display the real-time status (Processing, Shipped, Out for Delivery, Delivered).

4.2. Admin Dashboard

4.2.1. Check User Details

- **Scenario** – The admin checks the list of registered users.
- **Test Cases**
  - **Step 1** – Admin logs into the dashboard and navigates to the User Management section.
  - **Test Case 1** – Click "View User List".
  - **Result** – The system should display all users with details (ID, name, email, contact, status).

4.2.2. Update User Details

- **Scenario** – The admin updates a user's details.
- **Test Cases**
  - **Step 1** – Admin goes to User List and clicks the edit button on a user record.
  - **Test Case 1** – Modify fields like email/phone/status and click "Update".
  - **Result** – The system should save changes and display updated details in the list.

4.2.3. Delete User

- **Scenario** – The admin deletes a user from the system.
- **Test Cases**
  - **Step 1** – Admin navigates to User List.
  - **Test Case 1** – Click "Delete" beside a user.
  - **Result** – A confirmation popup should appear. Once confirmed, the user should be removed from the list.

4.2.4. Add Product

- **Scenario** – The admin adds a new product to the catalog.
- **Test Cases**
  - **Step 1** – Admin navigates to Product Management → Add Product.
  - **Test Case 1** – Fill in details (name, price, category, stock, warranty period, image) and click "Add Product".

- **Result** – The product should be added successfully and appear in the product list.

4.2.5. Check Product List

- **Scenario** – The admin checks all available products.
- **Test Cases**
  - **Step 1** – Admin navigates to Product List.
  - **Test Case 1** – View the table of products.
  - **Result** – The system should display all products with details (ID, name, stock, price, status).

4.2.6. Update Product Details

- **Scenario** – The admin updates product information.
- **Test Cases**
  - **Step 1** – Admin selects a product from the list and clicks edit.
  - **Test Case 1** – Change stock quantity, price, or description, then click "Update".
  - **Result** – The product details should update instantly in the product list.

4.2.7. Delete Product

- **Scenario** – The admin deletes an unwanted product.
- **Test Cases**
  - **Step 1** – Admin goes to Product List.
  - **Test Case 1** – Click "Delete" beside a product.
  - **Result** – The system should ask for confirmation and remove the product after approval.

4.2.8. Approve/Decline Warranty Claims

- **Scenario** – The admin manages customer warranty claims.
- **Test Cases**
  - **Step 1** – Admin navigates to Warranty Requests.
  - **Test Case 1** – Select a warranty request and click "Approve".
  - **Result** – Status changes to "Approved" and customer notified.
  - **Test Case 2** – Select a warranty request and click "Decline".

- **Result** – Status changes to "Declined" and customer notified.

4.2.9. Approve/Decline Repair Requests

- **Scenario** – The admin handles repair service requests.
- **Test Cases**
  - **Step 1** – Admin navigates to Repair Requests.
  - **Test Case 1** – Select a request and click "Approve".
  - **Result** – Status updates to "Approved" and customer notified.
  - **Test Case 2** – Select a request and click "Decline".
  - **Result** – Status updates to "Declined" and customer notified.

4.2.10. Update Order Status

- **Scenario** – The admin updates the delivery status of an order.
- **Test Cases**
  - **Step 1** – Admin navigates to Orders List.
  - **Test Case 1** – Select an order and update status (Processing → Shipped → Delivered).
  - **Result** – The system should reflect the new status and notify the customer.

4.2.11. View Order List

- **Scenario** – The admin checks all customer orders.
- **Test Cases**
  - **Step 1** – Admin navigates to Orders List.
  - **Test Case 1** – View orders table.
  - **Result** – The system should display all orders with details (Order ID, customer name, product, payment status, delivery status).

4.3. Repair and solution management

4.3.1. Submit Repair Request

- **Scenario** – A user submits a new repair request.
- **Test Cases**
  - **Step 1** – User logs in and navigates to the Repair Request form.
  - **Test Case 1** – Fill in required fields (product name, description, damaged image) and click "Submit".
  - **Result** – A new repair request is created with a unique sequential `repairId` (e.g., PR001), and stored in the database.

4.3.2. View own repair requests

- **Scenario** – A user views their submitted repair requests.
- **Test Cases**
  - **Step 1** – User navigates to "My Repairs".
  - **Test Case 1** – View list of repair requests.
  - **Result** – The system displays only the user's own requests with details (repairId, product, status, submitted date).

4.3.3. Update Repair Description or Images

- **Scenario** – A user updates the description or uploads images for a repair request they created.
- **Test Cases**
  - **Step 1** – User selects a repair request from "My Repairs" and clicks "Edit".
  - **Test Case 1** – Modify description or upload damaged/repaired images and click "Update".
  - **Result** – The system validates ownership and updates the request.

4.3.4. Search for repair requests

- **Scenario** – Admin searches repair requests using keywords.
- **Test Cases**
  - **Step 1** – Admin navigates to Repair Management → Search.
  - **Test Case 1** – Enter partial customer name or product name and click "Search".
  - **Result** – The system returns matching records using regex-based search.

4.3.5. Filter & Sort Repair Requests

- **Scenario** – Admin filters and sorts repair requests.
- **Test Cases**
  - **Step 1** – Admin navigates to Repair Management → Filter.
  - **Test Case 1** – Apply filters (status, date range, customer name, product name) and sort results.
  - **Result** – The system displays filtered and sorted repair requests.

4.3.6. View Repair Request Details

- **Scenario** – A user views detailed information about a specific repair request.
- **Test Cases**
  - **Step 1** – User navigates to "My Repairs" and clicks on a specific request.
  - **Test Case 1** – System displays full details (repairId, product name, description, status, images, timestamps).
  - **Result** – The user sees a complete view of their request, including any updates made by the admin.

4.3.7. Update Repair Status (Admin Only)

- **Scenario** – Admin updates the status of a repair request.

- **Test Cases**
  - **Step 1** – Admin selects a repair request and clicks "Update Status".
  - **Test Case 1** – Change status (Pending → In Progress → Completed) and click "Save".
  - **Result** – The system updates the status and notifies the user.

4.3.8. Admin filter by status

- **Scenario** – Admin filters repair requests based on their current status.
- **Test Cases**
  - **Step 1** – Admin selects a status filter (e.g., Pending, In Progress, Completed).
  - **Test Case 1** – Click "Apply Filter".
  - **Result** – System displays only requests matching the selected status.

4.3.9. Invalid Repair Request Submission

- **Scenario** – A user submits a repair request with missing or invalid data.
- **Test Cases**
  - **Step 1** – User leaves required fields blank or uploads unsupported image format.
  - **Test Case 1** – Click "Submit".
  - **Result** – System returns validation errors and prevents submission until corrected.

4.3.10. Unauthorized Update Attempt

- **Scenario** – A regular user attempts to update a field they don't have access to.
- **Test Cases**
  - **Step 1** – User tries to change the `status` field of a repair request.
  - **Test Case 1** – Submit update request.
  - **Result** – System blocks the action and returns an error message: "Unauthorized access.

4.4. Warranty Management

4.4.1. Submit Warranty Claim

- **Scenario** – A customer submits a warranty claim for repair or replacement.
- **Test Cases**
  - **Step 1** – The customer goes to *Submit Warranty Claim*.
  - **Test Case 1** – Enter claim details (issue description, product ID) and click submit.
  - **Result** – The system should generate a claim ID and mark status as *Pending*.

4.4.2. View Warranty Claim Status

- **Scenario** – A customer views the status of their submitted warranty claim.
- **Test Cases**
  - **Step 1** – The customer navigates to *My Warranty Claims*.
  - **Test Case 1** – Select a claim and view details.
  - **Result** – The system should display claim status (Pending, Approved, Declined, Completed).

4.4.3. Update or Cancel Warranty Claim

- **Scenario** – A customer updates or cancels an existing warranty claim.
- **Test Cases**
  - **Step 1** – The customer goes to *My Warranty Claims*.
  - **Test Case 1** – Select claim → Edit details or click *Cancel*.
  - **Result** – The system should save the update or mark the claim as *Cancelled*.

4.4.4. View Past Warranty Claims

- **Scenario** – A customer views their warranty claim history.
- **Test Cases**
  - **Step 1** – The customer navigates to *Warranty History*.
  - **Test Case 1** – Select the history tab.
  - **Result** – The system should display all past claims with details and status.

4.4.5. Submit Claim for Unregistered Product

- **Scenario** – A user tries to submit a claim for a product not registered under their account.
- **Test Cases**
  - **Step 1** – Navigate to "Submit Claim" and attempt to select an unregistered product.
  - **Test Case 1** – Try submitting the claim.
  - **Result** – The system should block the action and display an error.

4.4.6. Unauthorized Claim Edit Attempt

- **Scenario** – A user tries to edit a claim they don't own.
- **Test Cases**
  - **Step 1** – Access another user's claim via direct URL or manipulation.
  - **Test Case 1** – Attempt to edit the claim.
  - **Result** – The system should deny access and log the attempt.

4.4.7. Upload Invalid Proof of Purchase

- **Scenario** – A user uploads a corrupted or unsupported file format.
- **Test Cases**
  - **Step 1** – Upload a file in an unsupported format (e.g., .exe, .txt).
  - **Test Case 1** – Submit the form.
  - **Result** – The system should reject the file and prompt for a valid format (e.g., .jpg, .pdf).

Test case design

Test Cases for User Management Module (IT23614758)

Feature	Test Case ID	Test Case Name	Purpose	Preconditions	Steps	Expected Outcome	Test Data
User Registration	TCUM-001	New User Sign-Up	Verify successful registration	1. System is accessible 2. Email not previously registered	1. Click "Sign Up" 2. Enter valid details (name, email, password) 3. Submit	System: Creates new account Sends verification email	testuser@domain.com, Password123!
Authentication	TC-UM-002	User Login	Test valid credentials login	1. Registered test user exists Account is verified	1. Enter valid email/password 2. Click "Login"	Grants access: Displays dashboard Sets auth token	testuser@domain.com Password123!
Roles & Permissions	TC-UM-003	Admin Role Assignment	Test admin privileges	1.Super admin exists 2.New user created	1.Admin assigns "Admin" role New user	User can: Access admin panel Manage users	role=admin permissions=all

					logs in		

Roles & Permissions	TC-UM-004	Accountant Permissions	Verify rolebased access	1. Accountant role defined	1.Assign accountant role 2. Login as accountant	Can access: Transactions Reports Cannot access: User management	role=accountant
Profile Management	TC-UM-005	Update User Profile	Test profile editing	1. Test user logged in	1. Edit profile (phone, address) 2. Save changes	System: Updates profile Shows confirmation	phone=0712345678
User Deactivation	TC-UM-008	Deactivate User Account	Test account disabling	1. Active test user exists 2. Admin logged in	1. Admin deactivates user 2. User attempts login	Shows: "Account disabled" Prevents login	status=inactive

5. High Level system Diagram

Link – System Diagram

6. Innovation Parts of the Project

6.1. Stock update in real time

- One of the features of the E-commerce system is an automatic stock updating mechanism.
- When the admin adds new products, processes customer orders, or handles product returns, the system automatically updates the stock levels in real time.
- This ensures that the product availability shown to customers is always accurate, preventing overselling and improving the overall shopping experience.

6.2. Special feature

6.2.1. Chat box

- One of the features of the Mobile Electronics and Services system is a built-in chat box.
- This allows customers to directly interact with support staff, ask questions about products, track orders, and resolve issues in real time.

6.2.2. Google Map Integration

- A unique feature is the integration of Google Maps into the platform.
- Customers can easily locate nearby service centers or track delivery routes, ensuring better navigation and faster service accessibility.

6.2.3. OTP Verification

- The system includes OTP (One-Time Password) verification for enhanced security.
- During user registration or payment, an OTP is sent to the user's mobile/email, ensuring that only verified users can proceed.

6.2.4. JWT (JSON Web Token) Authentication

- For secure communication between client and server, the system uses JWT authentication.
- It helps validate user sessions without repeatedly storing login details, ensuring both efficiency and safety.

6.2.5. Role-Based Access Control

- The platform provides role-based access to different users.
- Admins, staff, and customers have different privileges, ensuring controlled and secure access to system features.

6.2.6. Encrypted Passwords

- Password security is maintained by using encryption technique.
- Even if the database is compromised, the actual passwords cannot be retrieved, safeguarding user accounts.

6.2.7. QR Code Integration

- QR codes are implemented for quick access to product details and warranty information.
- Customers can scan the code to instantly view specifications, service options, and claim details.

6.2.8. JSON Web Token (Advanced Session Security)

- The system uses JSON Web Tokens to manage sessions in a secure and scalable manner.
- This ensures that sensitive information is exchanged safely across devices without compromising performance.

7. Commercialization

7.1. Market Overview

- The system is primarily designed for mobile and electronics retailers, service providers, and customers who require a unified online platform to purchase devices, request repairs, and manage warranty claims.
    - **Trends:** The e-commerce industry is rapidly shifting towards mobile-first shopping, integrated digital payments, real-time order tracking, and personalized customer experiences. Consumers increasingly expect secure, fast, and reliable platforms to buy electronics and access after-sales services.
    - **Competition:** While existing e-commerce platforms (e.g., Amazon, Flipkart) dominate the market, most are broad marketplaces. This system differentiates itself by focusing specifically on **electronics, mobile devices, warranty claims, and repair services**, offering an end-to-end lifecycle solution from purchase to service.
    - **Market Demand:** There is a growing demand for specialized electronics platforms where customers can purchase devices, schedule repairs, and claim warranties from a single system. Businesses are also seeking digital solutions to manage orders, suppliers, and customer service more efficiently.

7.2. Value Proposition

- **Efficiency:** The platform streamlines the customer journey by integrating product browsing, order management, repair requests, and warranty claims into a single interface.

- **Cost Savings:** Customers save time and money by accessing both products and services in one place, while businesses reduce overheads through automated workflows (stock updates, role-based access, real-time order tracking).
- **Trust & Security:** With OTP verification, encrypted passwords, JWT authentication, and role-based access, the system ensures safe transactions and user data protection.
- **Convenience:** Customers can track orders, locate nearby service centers via Google Maps, and use QR codes to quickly access product or warranty details.
- **Scalability:** Designed to serve both small retailers and large service providers, the system can grow with demand, supporting new product categories and additional service features.

7.3. Revenue Model

- **Pricing Strategy – The platform will generate revenue through multiple streams:**
    - **Commission Model:** A percentage fee on each transaction (product purchase, repair request, warranty claim).
    - **Subscription-Based Plans:** Vendors and service centers pay monthly/annual fees to list products and services.
    - **One-Time Setup Fees:** For businesses requiring customized integration or branding of the platform.

- **Additional Revenue Streams:**
    - **Premium Features:** Advanced dashboards, AI-driven product recommendations, or analytics modules offered as paid add-ons.
    - **Advertising:** Vendors can promote their products or services through featured listings and sponsored ads.
    - **Training & Support:** Offering onboarding packages and digital marketing support for small vendors.

7.4. Product Growth

- **New Features (Planned Enhancements):**
    - AI-Powered Product Recommendations based on customer behavior and search history.
    - Integrated EMI / Buy Now Pay Later (BNPL) options for high-value electronics purchases.
    - Voice-enabled Chatbot for customer service and product queries.

- Mobile App Integration for real-time tracking and shopping convenience.
- Extended Service Marketplace for accessories, repairs, and insurance add-ons.

- **Improvement Plan:**

  - **User Feedback Integration:** Collecting customer/vendor input to improve usability.
  - **Market-Driven Upgrades:** Staying aligned with e-commerce trends like digital wallets, AR product previews, and instant delivery.
  - **Performance Optimization:** Regular updates to improve site speed, security, and transaction reliability.

7.5. Implementation Plan

- **Setup & Training**

  - Vendors will be guided through account creation, product uploads, and order tracking features.
  - Training resources (tutorials, webinars, and demo sessions) will ensure smooth adoption.
  - Customers will be provided with easy user guides for product browsing, payment, and service tracking.

- **Support & Maintenance**

  - Dedicated helpdesk via live chat, email, and phone.
  - Regular updates to support new payment gateways, devices, and warranty policies.
  - Proactive system monitoring to reduce downtime and technical issues.

7.6. Marketing Plan

- **Branding:** The platform will be positioned as a **trusted, electronics-focused e-commerce and service solution**, highlighting reliability, convenience, and security.
- **Sales Channels:**

  - **Digital Marketing:** Social media ads, influencer collaborations, and Google search campaigns targeting electronics buyers.
  - **Partnerships with Retailers & Service Centers:** Bringing local vendors and repair shops online.
  - **Mobile App Stores:** Promoting the platform through app downloads with offers and cashback campaigns.

7.6.1. Regulation and Compliance

- **E-commerce Standards:** The system will comply with consumer protection laws, online transaction policies, and vendor regulations.
- **Data Privacy & Security:**
  - Strong encryption for user data and payment information.
  - Compliance with GDPR and local data privacy laws.
  - Secure authentication with OTP, JWT, and role-based access.

7.7. Risk Management

- **Potential Risks:**
  - High competition from established e-commerce giants.
  - Resistance from traditional service providers unfamiliar with digital platforms.
  - Technical risks such as downtime, data breaches, or payment gateway failures.
- **Mitigation Strategies:**
  - Focused marketing highlighting specialized electronics + services features.
  - Training and onboarding support for vendors/service centers.
  - Strong cybersecurity framework with encrypted transactions and secure backups.

7.8. Financials

- **Revenue Goals:** Revenue will be driven by transaction commissions, vendor subscriptions, and premium services. Targets will be set for the first 3–5 years, aiming to capture a strong niche in the electronics sector.
- **Costs:**
  - Platform Development (website, mobile app, security, cloud hosting).
  - Marketing (digital ads, influencer partnerships, SEO campaigns).
  - Vendor & Customer Support (staff, onboarding, training).
  - Infrastructure (servers, backup, API integrations).
  - **Break-Even Point:** Expected within 2–3 years, depending on vendor adoption and transaction volume.

7.9. Partnerships

- Collaborations with electronics retailers, mobile service providers, and warranty partners.
- Technology alliances with payment gateway providers, cloud hosting services, and Google Maps API.
- Logistics partnerships with courier and delivery services for faster fulfillment.

7.10. Exit Plan

- **Future Options:**
  - Acquisition by larger e-commerce or electronics platforms.
  - Merger with logistics or service-focused startups for faster growth.
  - Long-term independence as a profitable, specialized marketplace in the electronics sector.

8. Individual Contribution

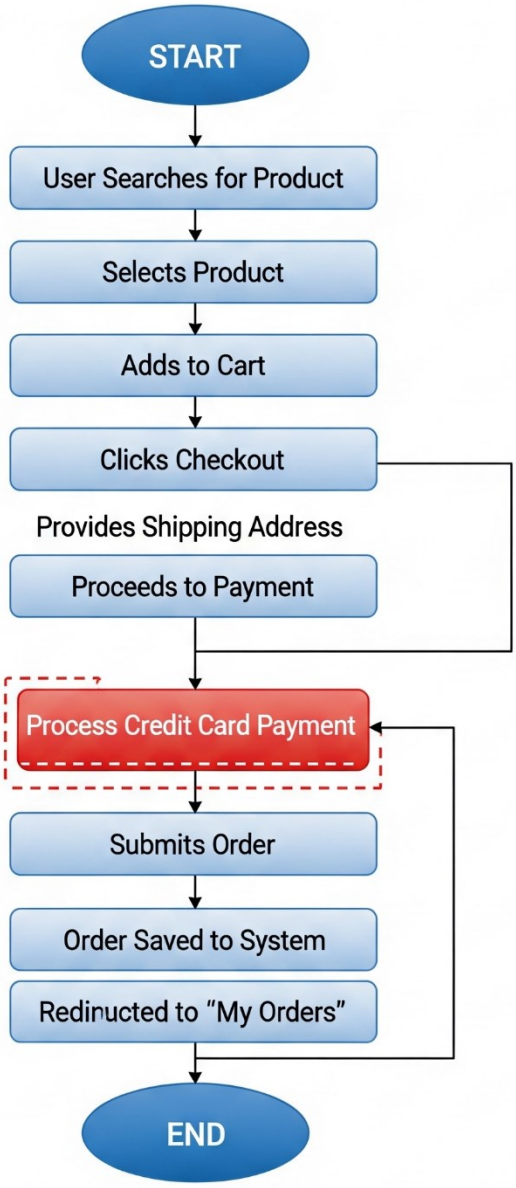
8.1. Order Management (IT23571884)

8.1.1. Features

Complete Features	Incomplete Features
Product Discovery & Cart ● Search for products. ● View product details. ● Add products to the shopping cart. ● View and update cart items. **Checkout Process** ● Add and select a shipping address.	**Payment Gateway** ● Integrate a third-party service (e.g., Stripe) to securely process credit card payments. **Real-time Notifications**

● Submit the final order. **Order History** ● View a list of past orders with their status and details.	● Implement email or push notifications to alert the user when their order status is updated by an admin.

8.1.2. Flow Chart



8.1.3. Pseudocode

```
FUNCTION main()
  BEGIN
    SearchForProduct()
    AddToCart()
    PlaceOrder()
    ViewOrderHistory()
  END
END FUNCTION


FUNCTION SearchForProduct()
  BEGIN
    searchQuery = GET_INPUT("Enter product name")
    CLICK_BUTTON("Search")
    IF (searchQuery IN searchResults) THEN
      DISPLAY "Showing results for: " + searchQuery
    ELSE
      DISPLAY "No products found."
    END IF
  END
END FUNCTION


FUNCTION AddToCart()
```

```
  BEGIN
    DISPLAY "Navigating to 'Product Details Page'."
    CLICK_BUTTON("Add to Cart")
    IF (product added to cart) THEN
      DISPLAY "Product successfully added to your cart."
    ELSE
      DISPLAY "Failed to add product to cart."
    END IF
  END
END FUNCTION


FUNCTION PlaceOrder()
  BEGIN
    DISPLAY "Navigating to 'Checkout Page'."
    shippingAddress = GET_INPUT("Enter your shipping address")
    CLICK_BUTTON("Confirm Address")


    DISPLAY "Navigating to 'Payment Page'."
    // Credit card integration is an incomplete feature


    CLICK_BUTTON("Submit Order")
    IF (order is successful) THEN
      DISPLAY "Your order has been placed successfully!"
```

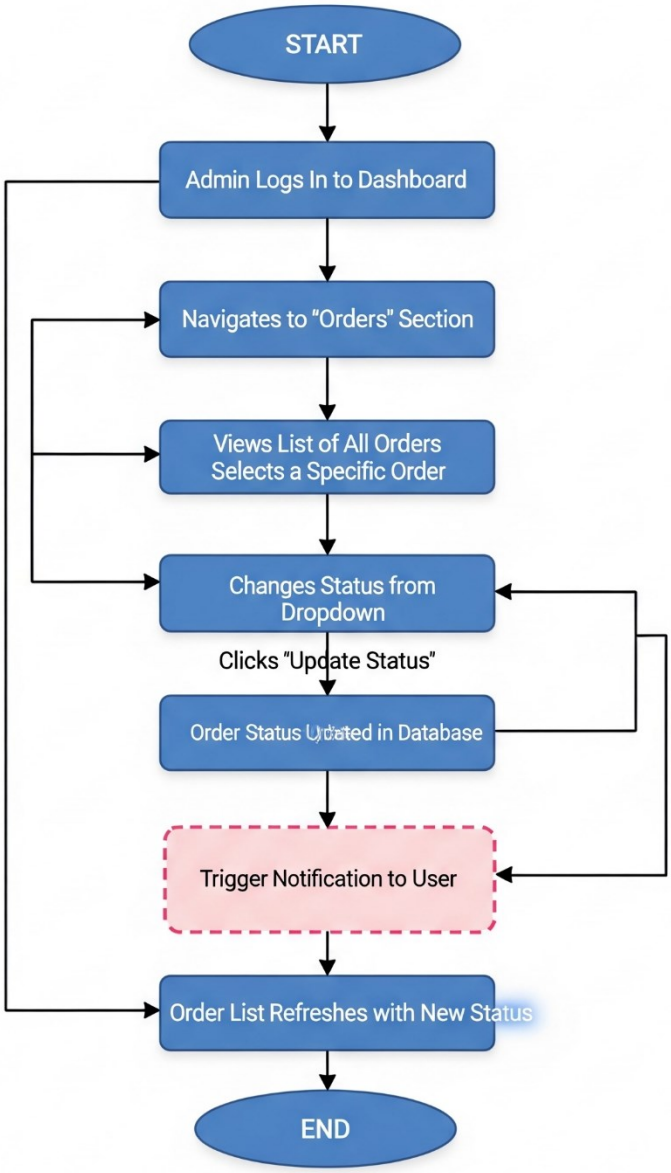
```
    ELSE
      DISPLAY "There was an error placing your order."
    END IF
  END
END FUNCTION


FUNCTION ViewOrderHistory()
  BEGIN
    DISPLAY "Navigating to 'My Orders Page'."
    DISPLAY orderList for the current user
  END
END FUNCTION
```

8.2. Admin Dashboard Management

8.2.1. Features

Complete Features	Incomplete Features
User Management ● View all registered users. ● Update user details. ● Delete users from the system. **Product Management** ● Add new products. ● View and edit existing product details. ● Delete products from the store. **Order Management** ● View a comprehensive list of all customer orders. ● Update the status of an order (e.g., "Processing," "Shipped," "Delivered"). ● Delete orders. **Services Approval** ● View user-submitted repair requests. ● Approve or deny repair requests. ● View user-submitted warranty claims. ● Approve or deny warranty claims.	**Sales Analytics** ● Design and implement dynamic graphs (e.g., bar chart, line graph) to visualize sales data over different time periods.

8.2.2. Flow Chart



8.2.1. Pseudocode

```
FUNCTION main()
 BEGIN
  AdminLogin()
  ManageUsers()
  ManageProducts()
  ManageOrders()
  ManageServiceRequests()
 END
END FUNCTION


FUNCTION ManageUsers()
 BEGIN
  DISPLAY "Navigating to 'User Management'."
  selectedUser = SELECT_FROM(userList)
  updatedDetails = GET_INPUT("Enter updated user details")
  CLICK_BUTTON("Update User")
  IF (details are updated) THEN
   DISPLAY "User updated successfully."
  END IF
 END
END FUNCTION
```

```
FUNCTION ManageProducts()
  BEGIN
    DISPLAY "Navigating to 'Product Management'."
    CLICK_BUTTON("Add New Product")
    productDetails = GET_INPUT("Enter name, price, description, etc.")
    CLICK_BUTTON("Save Product")
    IF (product is saved) THEN
      DISPLAY "Product added successfully."
    END IF
  END
END FUNCTION


FUNCTION ManageOrders()
  BEGIN
    DISPLAY "Navigating to 'Order Management'."
    selectedOrder = SELECT_FROM(orderList)
    newStatus = SELECT_FROM("Processing, Shipped, Delivered")
    CLICK_BUTTON("Update Status")
    IF (status is updated) THEN
      DISPLAY "Order status updated."
    END IF
  END
END FUNCTION
```

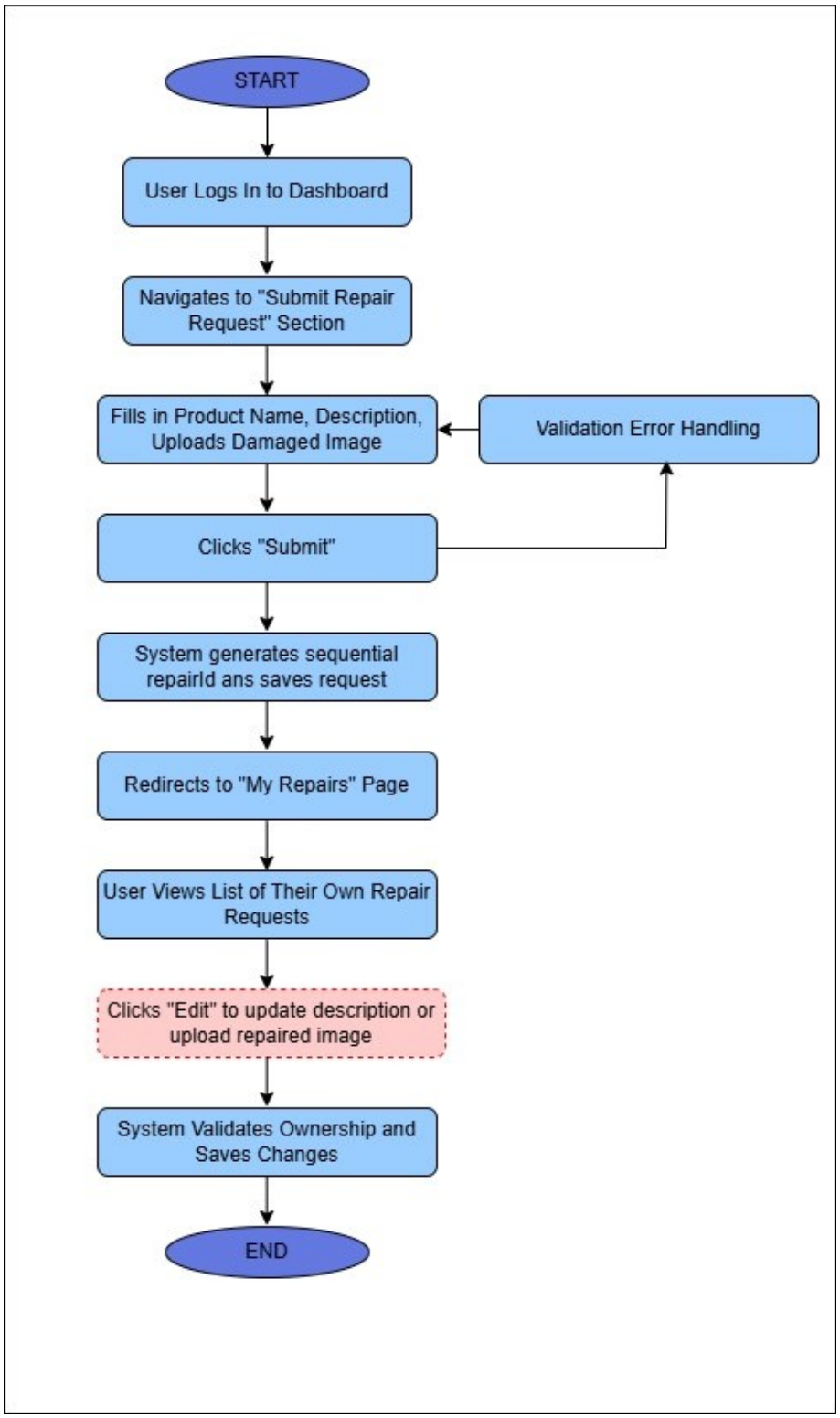
```
FUNCTION ManageServiceRequests()
  BEGIN
    DISPLAY "Navigating to 'Repair Requests'."
    selectedRequest = SELECT_FROM(repairRequestList)
    CLICK_BUTTON("Approve")
    IF (request is approved) THEN
      DISPLAY "Repair request approved and status updated."
    END IF
  END
END FUNCTION
```

8.3. Repair and solution Management (IT23542242)

8.3.1. Features

Complete Features	Incomplete Features
Repair Request Handling ● Submit new repair requests. ● View own repair requests with full details. ● Update description, damaged image, and repaired image. **Sequential ID Logic** ● Generates `repairId` based on existing entries. ● Maintains consistent format (e.g., PR001, PR002). ● Avoids duplication and supports easy tracking. **Admin Controls** ● View all repair requests submitted by users. ● Update repair status (Pending → In Progress → Completed) with role-based access control. ● Filter and sort repair requests by status, customer name, product name, and date range. ● Search repair requests using regex for partial matches.	**Frontend Interface** ● Build user-facing UI for submitting and viewing repair requests. ● Implement image upload components for damaged and repaired product photos. ● Create admin dashboard for managing and updating repair statuses.

8.3.2. Flow Chart



8.3.3. Pseudocode

```
FUNCTION main()
  BEGIN
    AdminLogin()
    ManageRepairRequests()
    UserSubmitRepair()
    UserUpdateRepair()
    SearchAndFilterRepairs()
  END
END FUNCTION


FUNCTION ManageRepairRequests()
  BEGIN
    DISPLAY "Navigating to 'Repair Management'."
    selectedRequest = SELECT_FROM(repairRequestList)
    action = SELECT_FROM("Approve, Decline, Update Status")
    IF (action == "Approve") THEN
      UPDATE_STATUS(selectedRequest, "Approved")
      DISPLAY "Repair request approved."
    ELSE IF (action == "Decline") THEN
      UPDATE_STATUS(selectedRequest, "Declined")
      DISPLAY "Repair request declined."
```

```
    ELSE IF (action == "Update Status") THEN
      newStatus = SELECT_FROM("Pending, In Progress, Completed")
      UPDATE_STATUS(selectedRequest, newStatus)
      DISPLAY "Status updated successfully."
    END IF
  END
END FUNCTION

FUNCTION UserSubmitRepair()
  BEGIN
    DISPLAY "Navigating to 'Submit Repair Request'."
    repairDetails = GET_INPUT("Enter product name, description, damaged image")
    GENERATE_REPAIR_ID()
    SAVE_TO_DATABASE(repairDetails)
    DISPLAY "Repair request submitted successfully."
  END
END FUNCTION

FUNCTION UserUpdateRepair()
  BEGIN
    DISPLAY "Navigating to 'My Repairs'."
    selectedRequest = SELECT_OWN_REQUEST()
    updatedFields = GET_INPUT("Update description or upload images")
```

```
    IF (userOwnsRequest) THEN
      UPDATE_REQUEST(selectedRequest, updatedFields)
      DISPLAY "Repair request updated."
    ELSE
      DISPLAY "Unauthorized access."
    END IF
  END
END FUNCTION

FUNCTION SearchAndFilterRepairs()
  BEGIN
    DISPLAY "Navigating to 'Search & Filter'."
    searchTerm = GET_INPUT("Enter keyword")
    filters = SELECT_FILTERS("Status, Date Range, Customer Name, Product Name")
    results = APPLY_SEARCH_AND_FILTER(searchTerm, filters)
    DISPLAY results
  END
END FUNCTION

FUNCTION FilterRepairRequests()
  BEGIN
    GET_FILTER_PARAMETERS(status, dateRange, customerName, productName)
    APPLY_FILTERS()
```

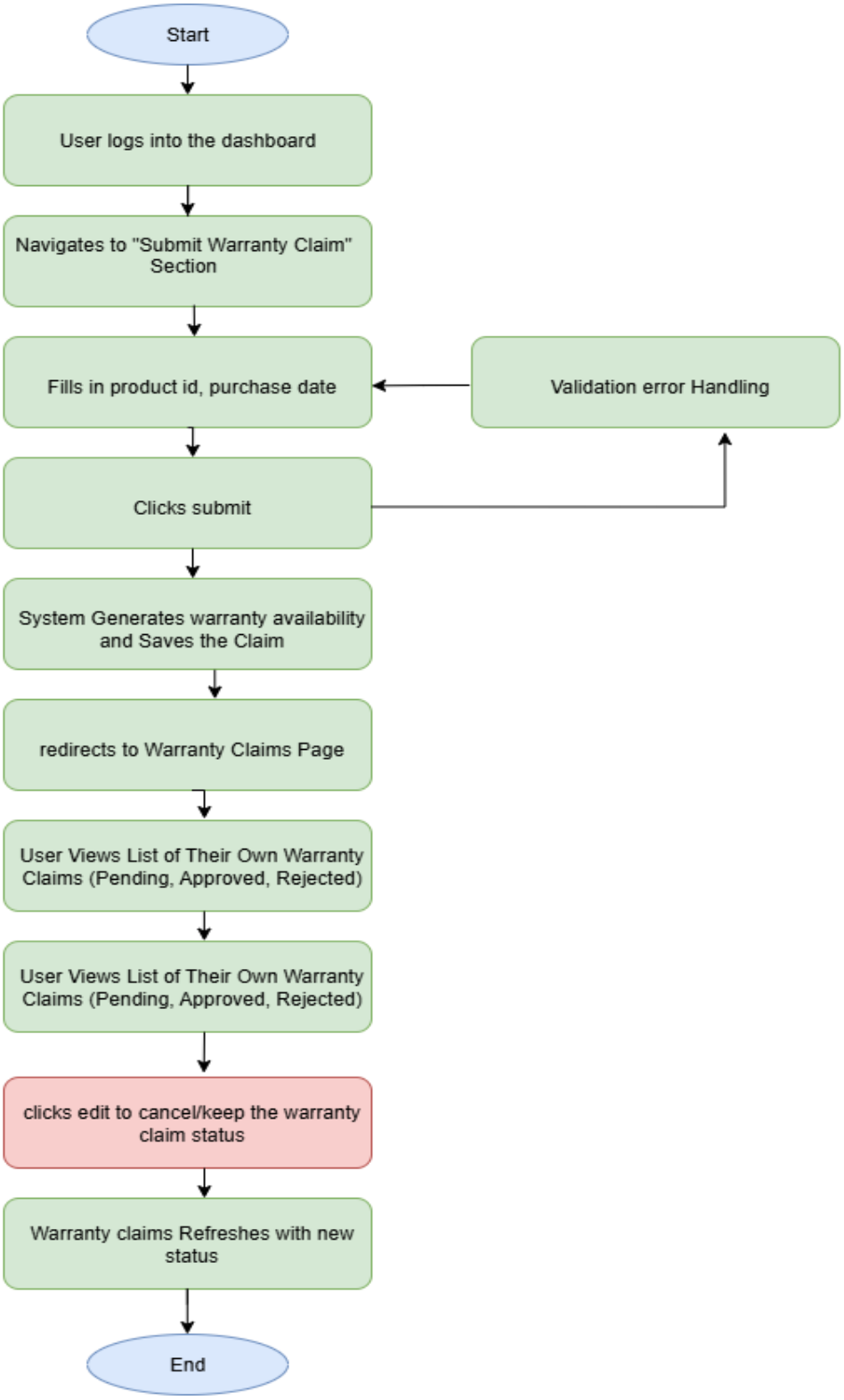
```
    SORT_RESULTS_IF_REQUESTED()
    RETURN FILTERED_LIST()
  END
END FUNCTION
```

8.4. Warranty Management (IT23540194)

8.4.1. Features

Complete Features	Incomplete Features
Warranty Claims • Submit a warranty claim request. • View the status of submitted claims (Pending, Approved, Declined, Completed). • Receive updates on warranty claim progress. **Claim History** • View a list of past warranty claims with their details and outcomes.	**Admin Review & Approval** • Enable admin to review and approve/reject warranty claims. **Admin Updates** • Allow admin to update claim details for accuracy. • Provide option for admin to mark a claim as completed once resolved. **Real-time Notifications** • Implement email or push notifications to alert customers when their warranty claim status is updated. .

8.4.2. Flowchart



8.4.3. Pseudocode

```
FUNCTION main()
  BEGIN
    LoginDashboard()
    NavigateToWarrantyClaimSection()
    SubmitWarrantyClaim()
    ViewWarrantyClaims()
    EditWarrantyClaim()
  END
END FUNCTION


FUNCTION LoginDashboard()
  BEGIN
    username = GET_INPUT("Enter your username")
    password = GET_INPUT("Enter your password")
    CLICK_BUTTON("Login")
    IF (login successful) THEN
      DISPLAY "Welcome to your Dashboard."
    ELSE
      DISPLAY "Invalid credentials. Please try again."
    END IF
  END
```

```
END FUNCTION


FUNCTION NavigateToWarrantyClaimSection()
  BEGIN
    DISPLAY "Navigating to 'Submit Warranty Claim' section."
  END
END FUNCTION


FUNCTION SubmitWarrantyClaim()
  BEGIN
    productID = GET_INPUT("Enter Product ID")
    purchaseDate = GET_INPUT("Enter Purchase Date")
    CLICK_BUTTON("Submit")


    IF (validation error occurs) THEN
      DISPLAY "Validation Error: Please check the inputs."
      // Optionally allow retry
      productID = GET_INPUT("Enter Product ID")
      purchaseDate = GET_INPUT("Enter Purchase Date")
      CLICK_BUTTON("Submit")
    END IF


    IF (warranty is available) THEN
```

```
      SAVE_CLAIM(productID, purchaseDate)
      DISPLAY "Warranty claim submitted successfully."
      REDIRECT_TO("Warranty Claims Page")
    ELSE
      DISPLAY "Warranty not available for this product."
    END IF
  END
END FUNCTION

FUNCTION ViewWarrantyClaims()
  BEGIN
    DISPLAY "Navigating to 'My Warranty Claims Page'."
    DISPLAY "Your Claims List: Pending, Approved, Rejected"
  END
END FUNCTION

FUNCTION EditWarrantyClaim()
  BEGIN
    claimID = GET_INPUT("Enter Claim ID to Edit")
    DISPLAY "Options: Cancel Claim / Keep Status"
    action = GET_INPUT("Select an action")

    IF (action == "Cancel Claim") THEN
```

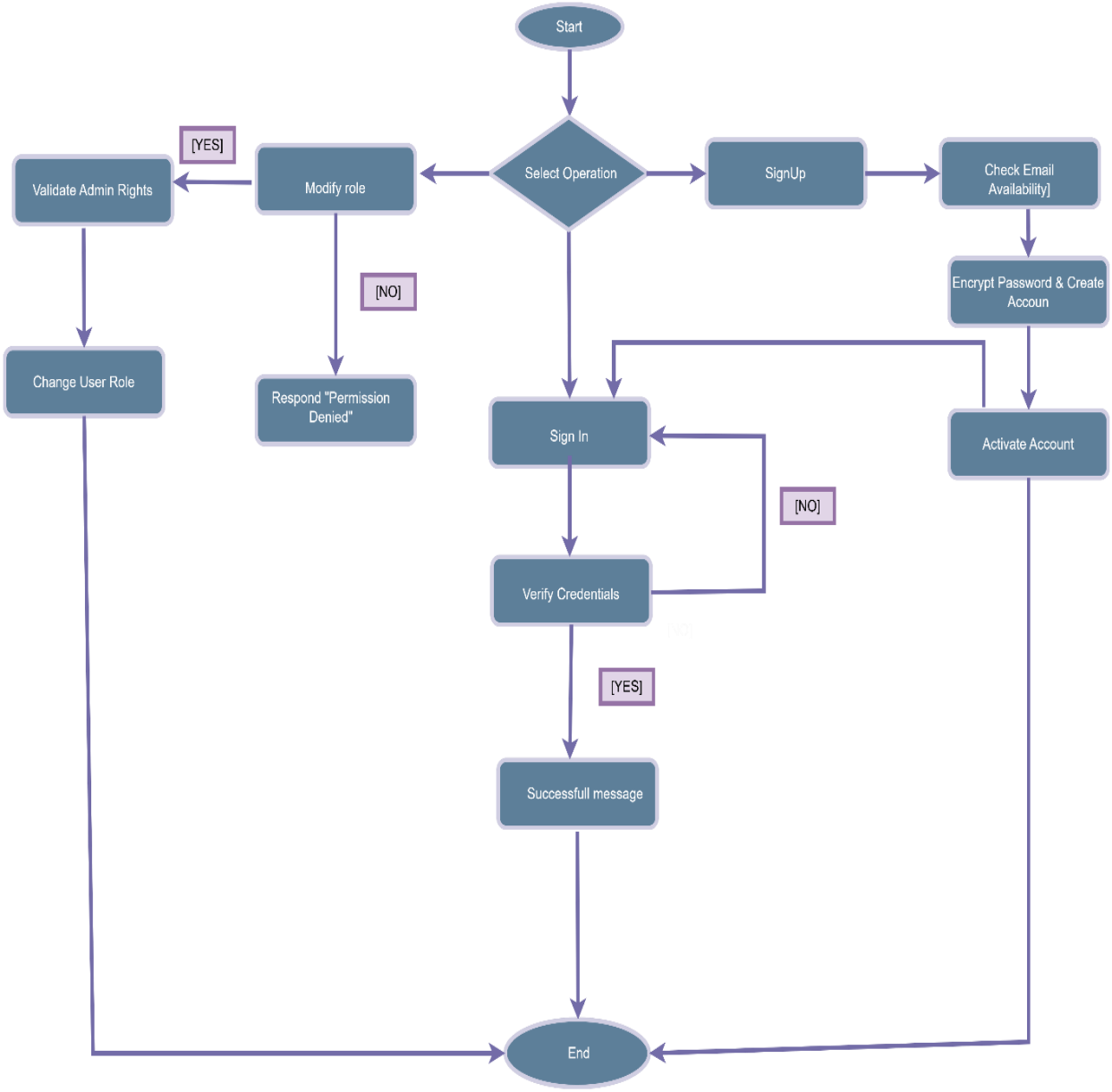
```
    UPDATE_CLAIM_STATUS(claimID, "Cancelled")
    DISPLAY "Warranty claim cancelled successfully."
  ELSE
    DISPLAY "Warranty claim status unchanged."
  END IF

  REFRESH_CLAIMS_PAGE()
 END
END FUNCTION
```

8.5 User Management (IT23614758)

8.5.1 Features

Complete Features	Incomplete Features
▪ User Registration ▪ User Login ▪ Password Reset & Recovery ▪ Role-Based Access Control ▪ Automated Financial Insights ▪Profile Management (update details, profile picture, preferences) ▪ Secure Data Encryption (for user info & transactions) ▪Session Management & Logout	▪ Risk Detection & Alerts ▪ AI Chat Assistant for Financial Queries ▪ Suggest ways to reduce costs and improve profit margins ▪ Guide users through tasks like uploading a bank statement. ▪Advanced Analytics & Forecasting.

8.4.4. Flowchart



8.5.3 Pseudocode

Pseudocode

```
FUNCTION manage_user(action, user_data)
  SWITCH action:
    CASE "register":
      IF user_data.email EXISTS IN database THEN
        THROW ERROR "Email already registered"
      ELSE
        SET hashed_password = HASH(user_data.password)
database.users.ADD({
          "name": user_data.name,
          "email": user_data.email,
          "password": hashed_password,
          "role": user_data.role
        })
        SEND_VERIFICATION_EMAIL(user_data.email)
      END IF

    CASE "update_role":
      IF current_user.role != "admin" THEN
        THROW ERROR "Unauthorized"
      ELSE
```

```
                    database.users.UPDATE(user_data.id, {"role":
user_data.new_role})
                END IF

            CASE "delete":
                database.users.REMOVE(user_data.id)

            DEFAULT:

THROW ERROR "Invalid action"
 END SWITCH
END FUNCTION
```

Thank you